Unit – 1 : DBMS Overview, SQL, SQL*Plus

Introduction to DBMS

DBMS stands for **Database Management System.**

A DBMS is software that helps users **create, store, organize, retrieve, and manage data** efficiently.

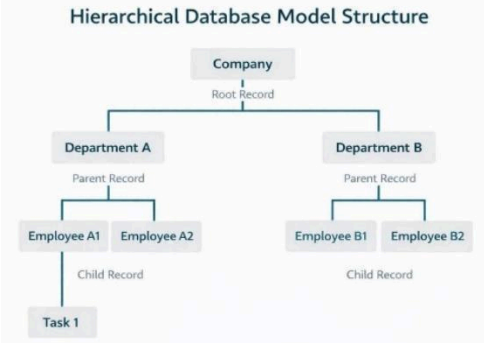
It acts as **an interface** between the database and the users or applications.

Functions of a DBMS

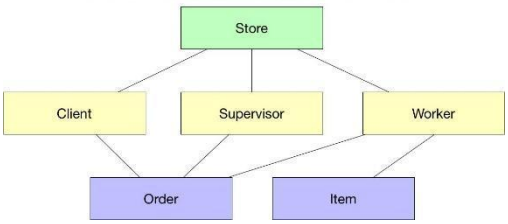
- Data storage and retrieval
- Data security and authorization
- Backup and recovery
- Data integrity maintenance
- Concurrency control (multiple users accessing data)
- Query processing

Types of DBMS

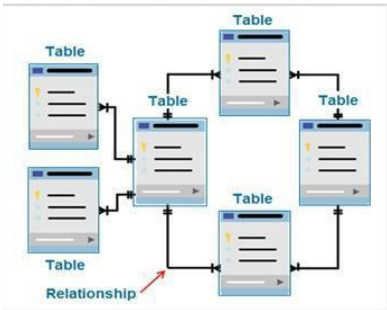
1. **Hierarchical DBMS**
   - Data is organized in a tree structure.
   - Each child has only one parent.
   - Fast for one-to-many relationships.
   - Example: IBM IMS



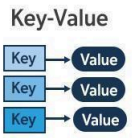
2. **Network DBMS**
   - Data is organized as a **network (graph)**.
   - Supports many-to-many relationships.
   - Allows multiple parent-child relationships.

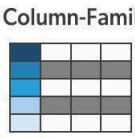


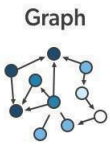
3. **Relational DBMS (RDBMS)**
   - Data is stored in tables (rows and columns).
   - Most widely used type.
   - Examples: MySQL, PostgreSQL, Oracle Database



4. **Object-Oriented DBMS (OODBMS)**
   - Data is stored as **objects**, similar to object-oriented programming.
   - Supports classes, objects, and inheritance.
   - **Example:** ObjectDB
5. **NoSQL DBMS**
   - Designed for large-scale and unstructured data.
   - Flexible schema and high performance.
   - Common in web and big-data applications.
   - Examples: MongoDB, Cassandra









Basic Components of a DBMS

1. **Database** – Collection of related data.
2. **DBMS Software** – Manages and controls the database.
3. **Users** – Database administrators, developers, and end-users.
4. **Applications** – Programs that interact with the database.

Introduction to RDBMS

RDBMS stands for Relational Database Management Systems.

All modern database management systems like SQL, MS SQL Server, IBM DB2, ORACLE, My-SQL and Microsoft Access are based on RDBMS.

It is called Relational Database Management System (RDBMS) because it is based on a relational model introduced by Dr. E. F. Codd. Data is represented in terms of tuples (rows) in RDBMS.

Relational database is the most commonly used database. It contains a number of tables and each table has its own primary key. Due to a collection of organized tables, data can be accessed easily in RDBMS.

DBMS	RDBMS
DBMS stands for **Database Management System**.	RDBMS stands for **Relational Database Management System**.
Data is stored as files or simple structures.	Data is stored in the form of related tables.
Relationships between data are not mandatory.	Relationships between tables are maintained using keys.
Supports small to medium-sized databases.	Supports large and complex databases.
Data redundancy is higher.	Data redundancy is minimized through normalization.
Security features are limited.	Provides better security and access control.
Usually supports a single user at a time.	Supports multiple users simultaneously.
Does not strictly follow the relational model.	Follows the relational model proposed by Edgar F. Codd.
Data integrity constraints are limited.	Enforces data integrity using Primary Key, Foreign Key, etc.
Examples: dBase, FoxPro.	Examples: MySQL, Oracle Database, Microsoft SQL Server.

Dr.E.F.Codd Rules:

E. F. Codd's rules were proposed by **Edgar F. Codd in 1985** to define what a true **Relational Database Management System (RDBMS)** should be.

Dr E.F.Codd, also known to the world as the '**Father of Database Management Systems**' had propounded 12 rules.

Before Codd's work, databases were difficult to manage and often relied on complex hierarchical or network structures.

He proposed storing data in **tables (relations)** and using mathematical principles to manage and query data efficiently.

The rules are numbered from zero to twelve. According to him, a DBMS is fully relational if it abides by all his twelve rules. Till now, only a few databases abide by all the eleven rules. His twelve rules are fondly called **'E.F.Codd's Twelve Commandments**'. His brilliant and seminal research paper 'A Relational Model of Data for Large Shared Data Banks' in its entirety is a visual treat to eyes. Here is brief note on E.F Codd's Twelve rules:

Rule 0 – Foundation rule: A system can be called an RDBMS only if it manages data using relational database principles.
Rule 1 : Information Rule: All data should be stored in **tables** (rows and columns). **Example:** Student details are stored in a Student table.
Rule 2 – Rule of Guaranteed Access: Every data item must be accessible by specifying Table name, Primary key (row identifier) or Column name. The use of pointers to access data logically is strictly forbidden. **Example:** Find a student's marks using their ID and the Marks column.
Rule 3 – Systematic Treatment of Null Values : Null values are completely supported in relational databases. They should be uniformly considered as '**missing information'**. Null values are **independent of any data type.** They should not be mistaken for blanks or zeroes or empty strings. Null values can also be interpreted as **'inapplicable data' or 'unknown information.' Example:** A student's phone number is not available yet.
Rule 4 – Data dictionary rule : or Active online catalog Rule : In the Database Management Systems **'metadata'** is **the data about the data**. Information about the database (tables, columns, etc.) should also be stored in the database called **'Data dictionary'. Example:** The database keeps a record of all table names and their structures.
Rule 5 – Common Language Rule : The system must support at least one relational language (such as SQL) for Data definition (DDL), Data manipulation (DML), Security, Integrity constraints or Transaction management.

Rule 6 – Rule of Updating Views: A view is a logical table which shows restricted data. Views generally make the data readable but not modifiable. Views help in data abstraction. if you create a virtual table (a "view") from your real tables, you should be able to change, delete, or add data directly through that view, as long as it is mathematically possible. The database system should handle the updates automatically.
Rule 7 – Rule of High-level insertion, update and deletion: A single operation should be sufficient to retrieve, insert, update and delete multiple data at a moment.
Rule 8 – Rule of Physical Data Independence: The data stored in a database must be independent of the applications that access the database. Changes to the physical storage methods or hardware (such as moving files to a different hard drive or changing indexing) must not require changes to the application programs or the logical structure of the database.
Rule 9 – Rule of Logical Data Independence: Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application.
Rule 10 – Rule of Integrity Independence: A database must be independent of the application that uses it. All its integrity constraints can be independently modified without the need of any change in the application. This rule makes a database independent of the front-end application and its interface.
Rule 11 – Rule of Distribution Independence: The end-user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only. This rule has been regarded as the foundation of distributed database systems.
Rule 12 – Rule of Non-Subversion: The non-subversion rule defines RDBMS as a SQL language to store and manipulate the data in the database. If a system has a low-level or separate language other than SQL to access the database system, it should not subvert or bypass integrity to transform data. Almost all full-scale DBMSs are RDMSs. Oracle implements 11+ rules and so does Sybase. SQL Server also implements 11+ rules while FoxPro implements 7+ rules.

Rule No.	Rule Name
0	Foundation Rule
1	Information Rule
2	Guaranteed Access Rule
3	Systematic Treatment of Null Values
4	Active Online Catalog
5	Comprehensive Data Sublanguage
6	View Updating Rule
7	High-Level Insert, Update, Delete
8	Physical Data Independence
9	Logical Data Independence
10	Integrity Independence
11	Distribution Independence
12	Non-Subversion Rule

[Extra : Integrity Constraints

An integrity constraint is a declarative method of defining a rule for a column of a table. Oracle supports the following integrity constraints:

- **NOT NULL** integrity constraints for the rules associated with nulls in a column
- **UNIQUE key** integrity constraints for the rule associated with unique column values
- **PRIMARY KEY** integrity constraints for the rule associated with primary identification values
- **FOREIGN KEY** integrity constraints for the rules associated with referential integrity. Oracle currently supports the use of FOREIGN KEY integrity constraints to define the referential integrity actions, including
    - update and delete RESTRICT
    - delete CASCADE

Schema : A database schema defines how **data is organized within a relational database**; this is inclusive of logical constraints such as, table names, fields, data types, and the relationships between these entities.**]**

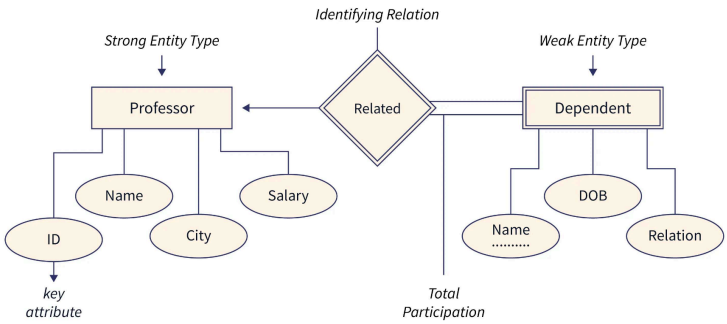
Importance of E. R. Diagram in Relational DBMS

- ER Diagram stands for **Entity Relationship Diagram,** also known **as ERD** is a diagram that displays the **relationship of entity sets** stored in a database.
- In other words, ER diagrams help to **explain the logical structure of databases**.
- It is used during the database design phase to model how data is stored and related before creating the actual database tables.
- ER diagrams are created based on three basic concepts: entities, attributes and relationships.
- ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes and diamond shape for relationships.
- It was proposed by **Peter Chen in 1971** to create a uniform convention which can be used for relational databases and networks.

Advantages of E-R Diagrams

- Simplifies database design.
- Provides a clear visual representation of data.
- Helps identify relationships between entities.
- Reduces data redundancy.
- Serves as a blueprint for creating database tables in an RDBMS.

Entity Relationship Diagram Symbols & Notations: An **Entity-Relationship (E-R) Diagram** is a **graphical representation of the entities, attributes, and relationships** in a database system. It is used during the database design phase to model how data is stored and related before creating the actual database tables. Mainly contains three basic symbols which are rectangle, oval and diamond to represent relationships between elements, entities and attributes.

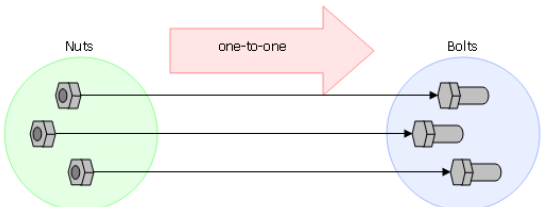


Components of an E-R Diagram

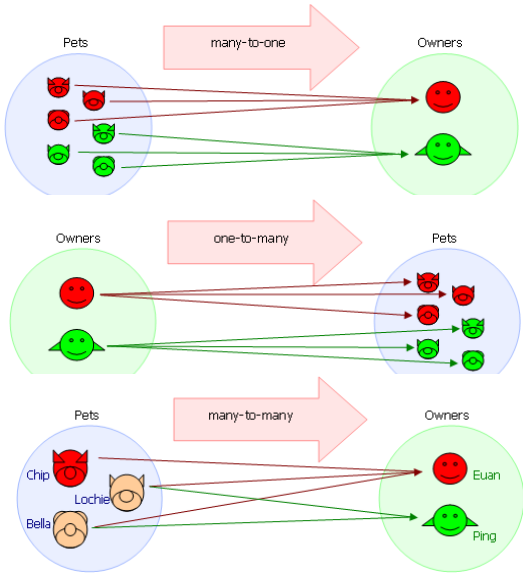
1. **Entity**
   - Represents a real-world object or concept about which data is stored.
   - An entity can be a place, person, object, event or a concept, which stores data in the database.
   - The characteristics of entities must have an attribute, and a unique key.
   - Shown using a **rectangle**.
   - Example: **Student**, **Course**, **Employee**.
2. **Attribute**
   - Describes the properties of an entity.
   - It is a single-valued property of either an entity-type or a relationship-type. For example, a lecture might have attributes: time, date, duration, place, etc.
   - Shown using an **oval (ellipse)**.
   - Example: Student_ID, Name, Address.
3. **Relationship**
   - Shows how entities are associated with each other.
   - We can often identify relationships with verbs or verb phrases.
   - Shown using a **diamond**.
   - Example: A **Student** *enrolls in* a **Course**.

Understand the different types of relationships

1. **One-to-One (1:1) :**



A one-to-one relationship is where one entity instance interacts only with one other entity instance.
Entity 1: the nut Entity 2: the bolt



2. **One-to-Many (1:M) :**

A one-to-many relationship is where one entity instance interacts with many other entity instances. This is the traditional hierarchical model relationship.
Example : One **Department** has many **Employees**.
Each **Employee** works in only one **Department**.

3. **Many-to-One (M:1) :**

A many-to-one relationship is where many entity instances belong to only one entity instance. This is the reverse of a one-to-many relationship.
Example : Many **Students** belong to one **Class**.
4. Many-to-Many M:N : A many-to-many relationship is where multiple instances can interact with many other entity instances.
Example : A **Student** can enroll in many **Courses**.
A **Course** can have many **Students**.

Normalization [EXAMPLE] [VIDEO LINK]
Normalization is the process of organizing the data in the database. Normalization is used to minimize the redundancy from a relation or set of relations.
Why is Normalization Important?

- **Reduces data redundancy** (avoids storing the same data repeatedly).
- **Improves data consistency**.(ensures that **all copies of data in the database have the same correct value** and there are no contradictions.)
- **Prevents update anomalies** (problems during insert, update, or delete operations).
- **Makes the database more efficient and easier to maintain.**
- **Improves data integrity.** (data integrity ensures that the data stored in the database is correct, complete, and trustworthy.)

Advantages of Normalization

- Eliminates duplicate data.
- Saves storage space.
- Ensures accurate and consistent data.
- Simplifies database maintenance.
- Reduces insertion, deletion, and update anomalies.

First Normal Form (1NF):

- 1NF ensures that each column contains only one value (atomic value - **atomic value** means **"one cell contains only one value."**).
- A table is in **1NF** if every cell contains a single value, and there are no repeating groups or multiple values in a column.
- It is made Easy to search data, Better organization of your data and No repeating groups in your table.

Suppose we have the following student table:

Student_ID	Student_Name	Subjects
101	Amit	DBMS, Java
102	Priya	Python, DBMS

What's Wrong?

- The **Subjects** column contains multiple values.
- This makes searching and updating data difficult.

Convert to 1NF

Student_ID	Student_Name	Subject
101	Amit	DBMS
101	Amit	Java
102	Priya	Python
102	Priya	DBMS

We rearrange the relation (table) as below, to convert it to First Normal Form.

Second Normal Form (2NF):

- 2NF **removes partial dependency** from the table.
- A table is in **2NF** if It is already in **1NF,** All non-key attributes depend on the **entire primary key**.
- **Partial Dependency :** Partial dependency occurs when an attribute depends on **only part of a composite primary key**, not the whole key.
- **Rule of 2NF:** Non-key attributes should depend on the **whole primary key**.

Benefits:

- Reduces duplicate data
- Eliminates partial dependency

Example

Primary Key = (Student_ID, Course_ID)

Student_ID	Course_ID	Student_Name	Course_Name
101	C01	Amit	DBMS
101	C02	Amit	Java
102	C01	Priya	DBMS

Problem:

- **Student_Name** depends only on **Student_ID**
- **Course_Name** depends only on **Course_ID**

They do not depend on the complete key (Student_ID + Course_ID).
This is called **Partial Dependency**.

Convert to 2NF (Student Table)

Student_ID	Student_Name
101	Amit
102	Priya

Course Table

Course_ID	Course_Name
C01	DBMS
C02	Java

Enrollment Table :

Student_ID	Course_ID
101	C01
101	C02
102	C01

Third Normal Form (3NF):

- 3NF removes transitive dependency from the table.
- A table is in 3NF if It is in **2NF and** Non-key attributes depend **only on the primary key**, not on other non-key attributes.
- A non-key attribute is **information about an entity that is not part of the primary key**.
- **for example** : Student_ID → Primary Key (uniquely identifies each student)
- Student_Name, Age, City → Non-Key Attributes
- These attributes describe the student but **cannot uniquely identify** a student.

What is Transitive Dependency?

Primary Key → Non-Key Attribute → Another Non-Key Attribute

Rule for #nf is Non-key attributes should depend **only on the primary key**.

Benefits of 3NF:

- Removes unnecessary dependencies
- Improves data integrity
- Reduces redundancy

Example :

Student_ID	Student_Name	Department_ID	Department_Name
101	Amit	D01	Computer Science
102	Priya	D02	Information Technology

Problem:

- Student_ID → Department_ID
- Department_ID → Department_Name

Therefore:

Student_ID → Department_Name

Department_Name depends indirectly on Student_ID.

This is called **Transitive Dependency**.

Convert to 3NF

Student Table

Student_ID	Student_Name	Department_ID
101	Amit	D01
102	Priya	D02

Department Table

Department_ID	Department_Name
D01	Computer Science
D02	Information Technology

Boyce Codd normal form (BCNF):

- BCNF is a **stronger version of 3NF**.
- A table is in **BCNF** if for every functional dependency: A → B **(A must be a candidate key.)**
- A **candidate key** is an attribute (or set of attributes) that can uniquely identify a record.

Rule of BCNF: Every determinant must be a candidate key.

Benefits:

- Removes all remaining anomalies
- Provides a stronger normalization than 3NF
- Ensures better database consistency

Example :

- One teacher teaches only one subject.
- One subject can be taught in many classrooms.

Teacher	Subject	Classroom
Shah	DBMS	A101
Shah	DBMS	A102
Patel	Java	B201

Functional Dependency:

Teacher → Subject

Because each teacher teaches only one subject.

However, the candidate key is:

Teacher + Classroom

Since **Teacher** alone is not a candidate key, the table violates BCNF.

Convert to BCNF

Teacher_Subject Table

Teacher	Subject

Shah	DBMS
Patel	Java

Subject_Classroom Table

Subject	**Classroom**
DBMS	A101
DBMS	A102
Java	B201

Introduction to SQL

SQL stands for Structured Query Language.
It is designed for **managing data** in a relational database management system (RDBMS).
In simple words, **SQL is the language that allows us to ask questions to a database and perform operations on the data it contains.**
SQL is important because it acts as a **bridge between users and databases.**
Almost every application that stores data, such as banking systems, e-commerce websites, hospital management systems, and educational portals, uses SQL to manage information.
SQL is a database language, it is used for database creation, deletion, fetching rows, and modifying rows, etc.
SQL works with **Relational Database Management Systems (RDBMS)** such as MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, and SQLite.

Uses of SQL

- **Store Data:** Insert new records into database tables.
- **Retrieve Data:** Fetch required information from databases.
- **Update Data:** Modify existing records.
- **Delete Data:** Remove unwanted records.
- **Create Database Objects:** Create tables, views, and indexes.
- **Control Access:** Grant or revoke permissions to users.
- **Generate Reports:** Analyse and summarize data for decision-making.

SQL Commands/Statements :
SQL commands are instructions used to communicate with a database to perform different operations such as creating tables, inserting data, retrieving data, updating records, and controlling access to the database.
SQL statements are started with any of the SQL commands/keywords like SELECT, INSERT, UPDATE, DELETE, ALTER, DROP etc. and the statement ends with a semicolon (;).

Some rules for SQL Commands :

- SQL is not case sensitive.
- Generally SQL keywords are written in uppercase. SQL statements are dependent on text lines.
- We can place a single SQL statement on one or multiple text lines.
- You can perform most of the action in a database with SQL statements.
- SQL depends on relational algebra and tuple relational calculus.

The following are the category of SQL Statements:

Category	**Full Form**	**Purpose**	**Commands**
DDL	Data Definition Language	Defines database structure	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Manipulates data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieves data	SELECT
DCL	Data Control Language	Controls permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions	COMMIT, ROLLBACK, SAVEPOINT

Data Definition Language (DDL): SQL uses the following set of commands to define database schema:

- **CREATE:** Creates new databases, tables and views from RDBMS.
- **DROP:** Drops commands, views, tables, and databases from RDBMS.
- **ALTER:** Modifies database schema.

Data Manipulation Language (DML): SQL is equipped with data manipulation language (DML). DML modifies the database instance by inserting, updating and deleting its data. DML is responsible for all forms of data modification in a database. SQL contains the following set of commands in its DML section:

- **INSERT:** This command is used for inserting values into the rows of a table (relation).
- **UPDATE:** This command is used for updating or modifying the values of columns in a table (relation).
- **DELETE:** This command is used for removing one or more rows from a table (relation).

Data Query Language (DQL): SELECT : DQL commands are used to **retrieve data from the database**. This is one of the fundamental query commands of SQL.

Transaction Control Language (TCL): Transaction Control Language commands are used to manage transactions in the database. These are used to manage the changes made by DML-statements. It also allows statements to be grouped together into logical transactions.

- **COMMIT:** Commit command is used to permanently save any transaction into the database.
- **ROLLBACK:** This command restores the database to the last committed state. It is also used with the savepoint command to jump to a savepoint in a transaction.
- **SAVEPOINT:** Savepoint command is used to temporarily save a transaction so that you can roll back to that point whenever necessary.

Data Control Language (DCL): A Data Control Language is a syntax similar to a computer programming language used to control access to data stored in a database (Authorization). In particular, it is a component of Structured Query Language (SQL).

- **GRANT:** allow specific users to perform specified tasks.
- **REVOKE:** cancel previously granted or denied permissions.

SQL Data Types

Data types are used to represent the nature of the data that can be stored in the database table.
For example, in a particular column of a table, if we want to store a string type of data then we will have to declare a string data type of this column. Data types are mainly classified into three categories for every database.

1. Numeric Data Types : Used to store numeric values.

Data Type	Description	Example
INT	Stores whole numbers	10, 250, -45
SMALLINT	Stores smaller range integers	1000
BIGINT	Stores very large integers	9876543210
DECIMAL(p,s)	Stores exact decimal values	1234.56
NUMERIC(p,s)	Similar to DECIMAL	45.75
FLOAT	Stores approximate floating-point numbers	3.14159
REAL	Single precision floating-point number	15.78
DOUBLE	Double precision floating-point number	12345.6789

2. Character/String Data Types : Used to store text data.

Data Type	Description	Example
CHAR(n)	Fixed-length character string	CHAR(10)
VARCHAR(n)	Variable-length character string	VARCHAR(50)
TEXT	Stores large text data	Paragraphs, descriptions
NCHAR(n)	Fixed-length Unicode characters	NCHAR(20)
NVARCHAR(n)	Variable-length Unicode characters	NVARCHAR(100)

Difference between Char v/s Varchar :

CHAR	VARCHAR
Fixed size	Variable size
Wastes space if data is shorter	Saves storage space
Faster for fixed-length data	More flexible

3. Date and Time Data Types : Used to store date and time values.

Data Type	Description	Example
DATE	Stores date only	'2026-06-13'
TIME	Stores time only	'10:30:45'
DATETIME	Stores both date and time	'2026-06-13 10:30:45'
TIMESTAMP	Stores date and time with automatic updates	'2026-06-13 10:30:45'
YEAR	Stores year values	2026

4. Boolean Data Type : Used to store logical values.

Data Type	Description	Values
BOOLEAN	Stores TRUE or FALSE values	TRUE, FALSE
BIT	Stores binary logical values	0, 1

5. Binary Data Types : Used to store binary objects such as images, videos, and files.

Data Type	Description
BINARY(n)	Fixed-length binary data
VARBINARY(n)	Variable-length binary data
BLOB	Binary Large Object

6. Large Object Data Types :

Data Type	Description
TEXT	Large amount of character data
BLOB	Large amount of binary data
CLOB	Character Large Object

Difference Between BLOB and CLOB

BLOB	CLOB
Stores binary data	Stores character data
Images, videos	Documents, articles
Cannot directly read as text	Readable text

Introduction to SQL*Plus

- SQL*Plus is a tool used to communicate with an Oracle database using SQL commands.
- PL/SQL stands for “Procedural Language extensions to SQL.” PL/SQL is a database-oriented programming language that extends SQL with procedural capabilities.
- It is used to execute SQL statements, PL/SQL programs, and perform database management tasks.
- It provides a command-line environment for managing and accessing Oracle databases.
- It contains its own command language.
- It accesses local & remote databases.
- It’s an interactive user interface to the ORACLE database management system.
- Typically, SQL *PLUS is used to issue queries & to view the query results on the screen.

Uses of SQL*Plus (Explain features of SQL*)

1. **Executing SQL Commands :** Used to run commands like SELECT, INSERT, UPDATE, and DELETE.
2. **Running PL/SQL Programs :** Executes PL/SQL blocks and stored procedures.
3. **Creating and Managing Database Objects :** Create, modify, and delete tables, views, indexes, etc.
4. **Retrieving Data :** Fetches data from database tables using SQL queries.

5. **Generating Reports :** Formats and displays query results in a readable form.
6. **Running SQL Script Files :** Executes saved SQL commands from script files.
7. **Database Administration :** Helps database administrators manage Oracle databases.

DIFFERENCE BETWEEN SQL AND SQL * PLUS

SQL	SQLPLUS
SQL (Structured Query Language) is a database language.	SQL*Plus is an Oracle utility/tool.
Used to create, retrieve, update, and delete data in a database.	Used to execute SQL and PL/SQL commands.
It is a standard language used in many databases.	It is mainly used with Oracle databases.
SQL commands are processed by the database server.	SQL*Plus provides an interface between the user and the Oracle database.
Examples: SELECT, INSERT, UPDATE, DELETE.	Examples: CONNECT, DESC, SPOOL, EXIT.
SQL focuses on data manipulation and definition.	SQL*Plus focuses on executing commands, formatting output, and running scripts.
SQL can be embedded in PL/SQL.	PL/SQL can't be embedded in SQL.
There are 4 types of SQL commands like DDL DML TCL DCL	There are 6 types of SQL*commands like : SQL * Plus Execute commands SQL * Plus Editing commands SQL * Plus Formatting commands Miscellaneous commands Access commands for various databases. File manipulation commands
SQL does not provide error and exception handling.	PL/SQL provides error and exception handling.

SQL*PLUS FORMATTING COMMAD

SQL*Plus Formatting Commands are **used to format the display** of query results. These commands make the output more readable and professional without changing the actual data stored in the database. They are especially useful **when generating reports.**

Command	Purpose	Short Example
COLUMN	Formats a column width	COLUMN ename FORMAT A15 (Displays employee names in a field of 15 characters.)
HEADING	Changes column heading	COLUMN ename HEADING 'Employee Name' (Changes the heading from ENAME to Employee Name.)
FORMAT	Formats numeric values	COLUMN sal FORMAT 99,999 (Displays salary with commas (e.g., 25,000).)
SET LINESIZE	Sets width of output line	SET LINESIZE 100 (Allows 100 characters per output line)
SET PAGESIZE	Sets number of rows per page	SET PAGESIZE 50 (Displays 50 rows before a page break)
TTITLE	Displays title at top of report	TTITLE 'EMPLOYEE REPORT' (Prints "EMPLOYEE REPORT" at the top)

BTITLE	Displays title at bottom of report	BTITLE 'END OF REPORT' (Prints "END OF REPORT" at the bottom)
SPOOL	Saves output to a file	SPOOL report.txt (Saves query output into report.txt.)

Note that none of these SQL*Plus formatting commands changes the underlying table structures.

SQL Operator

An operator is a symbol which represents particular operations that can be performed on some data.

Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values.

Operator	Meaning	Example
+	Addition	Salary + 1000
-	Subtraction	Salary - 500
*	Multiplication	Quantity * Price
/	Division	Total / 2
%	Modulus (Remainder)	10 % 3

Example:

SELECT 10 + 5 AS Result;

Output: 15

SELECT Salary * 12 AS Annual_Salary FROM Employee;

Comparison (Relational) Operators

Comparison operators compare two values and return TRUE or FALSE.

Operator	Meaning
=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Examples:

Equal To (=) : *SELECT * FROM Student WHERE City = 'Rajkot';*
Greater Than (>) : *SELECT * FROM Student WHERE Marks > 75;*
Not Equal To (<>): *SELECT * FROM Student WHERE City <> 'Rajkot';*

Logical Operators :

Logical operators combine multiple conditions.

Operator	Meaning
AND	All conditions must be TRUE
OR	At least one condition must be TRUE
NOT	Reverses the condition

AND Operator : SELECT * FROM Student WHERE Marks > 70 AND City='Rajkot';
OR Operator : SELECT * FROM Student WHERE City='Rajkot' OR City='Ahmedabad';
NOT Operator : SELECT * FROM Student WHERE NOT City='Rajkot';

Special Operators

1. BETWEEN Operator : Used to find values within a range.

SELECT * FROM Student WHERE Marks BETWEEN 60 AND 80;
Meaning: Marks from 60 to 80 (inclusive).

2. IN Operator : Used to specify multiple values.

SELECT * FROM Student WHERE City IN ('Rajkot','Surat','Ahmedabad');
Equivalent to: WHERE City='Rajkot', OR City='Surat', OR City='Ahmedabad';

3. LIKE Operator : Used for pattern matching.

Symbol	Meaning
%	Any number of characters
_	Single character

SELECT * FROM Student WHERE Name LIKE 'A%';
Names starting with A.
SELECT * FROM Student WHERE Name LIKE '_a%';
Second letter is 'a'.

4. IS NULL Operator : Used to find NULL (empty) values.
SELECT * FROM Student WHERE Phone IS NULL;

5. EXISTS Operator : Checks whether a subquery returns any records.
SELECT * FROM Student WHERE EXISTS (SELECT * FROM Result WHERE Student.ID = Result.ID);

6. **ANY OPERATOR :** These operator compares a value to each value returns by sub query.

SELECT ename, job FROM EMP WHERE sal < **ANY (**select sal from EMP where deptno=20) **;**

7. **ALL OPERATORS :** These operator compares a value to every value returns by sub query.

SELECT ename, job FROM EMP WHERE sal < **ALL (**select sal from EMP where job='MANAGER') **;**

Set Operators
Used to combine results from multiple SELECT statements.

1. UNION : Combines results and removes duplicates.
SELECT City FROM Student UNION SELECT City FROM Employee;

2. UNION ALL : Combines results including duplicates.
SELECT City FROM Student UNION ALL SELECT City FROM Employee;

3. INTERSECT Returns common records.
SELECT City FROM Student INTERSECT SELECT City FROM Employee;

4. MINUS (EXCEPT) : Returns records from first query not found in second.
SELECT City FROM Student MINUS SELECT City FROM Employee;

SQL Expression

- An SQL expression is a formula used in an SQL statement to calculate, compare, or manipulate data and return a value.
- A simple definition of expression is that: An expression returns a value.

Types of SQL Expressions

1. **Arithmetic Expressions** (+, -, *, /)
2. **Comparison Expressions** (=, <, >, <=, >=, <>)
3. **Logical Expressions** (AND, OR, NOT)
4. **Character/String Expressions**
5. **Function Expressions** (UPPER, LOWER, COUNT, SUM, etc.)

Examples :

1. Arithmetic Expression
SELECT salary + 5000 FROM employee;
Here, salary + 5000 is an expression.

2. Comparison Expression
SELECT * FROM employee
WHERE salary > 30000;
Here, salary > 30000 is an expression.

3. String Expression
SELECT first_name || ' ' || last_name FROM employee;

This combines two strings into one.

4. Function Expression

SELECT UPPER(name) FROM student;

UPPER(name) is an expression that converts text to uppercase.